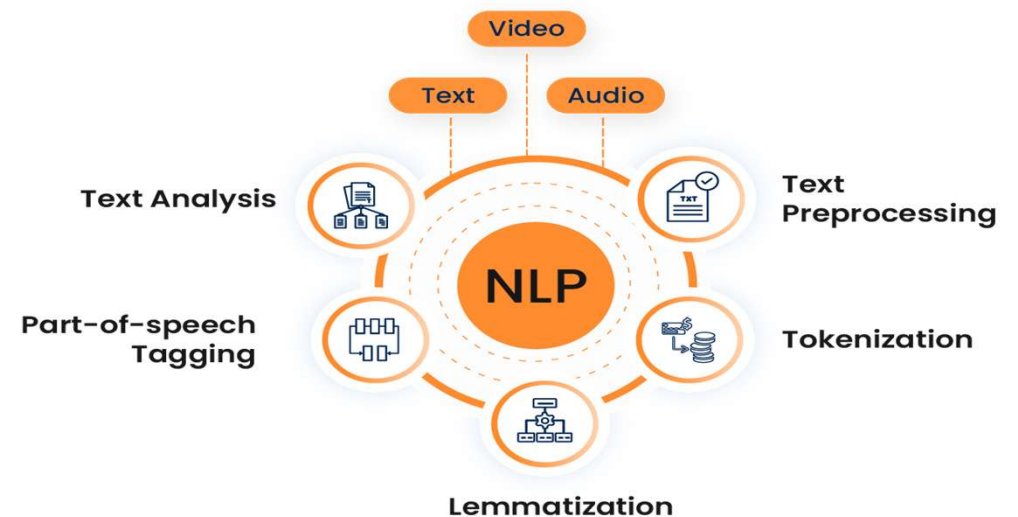


Transformers LLM

- Varun Raste
Data Scientist , IBM

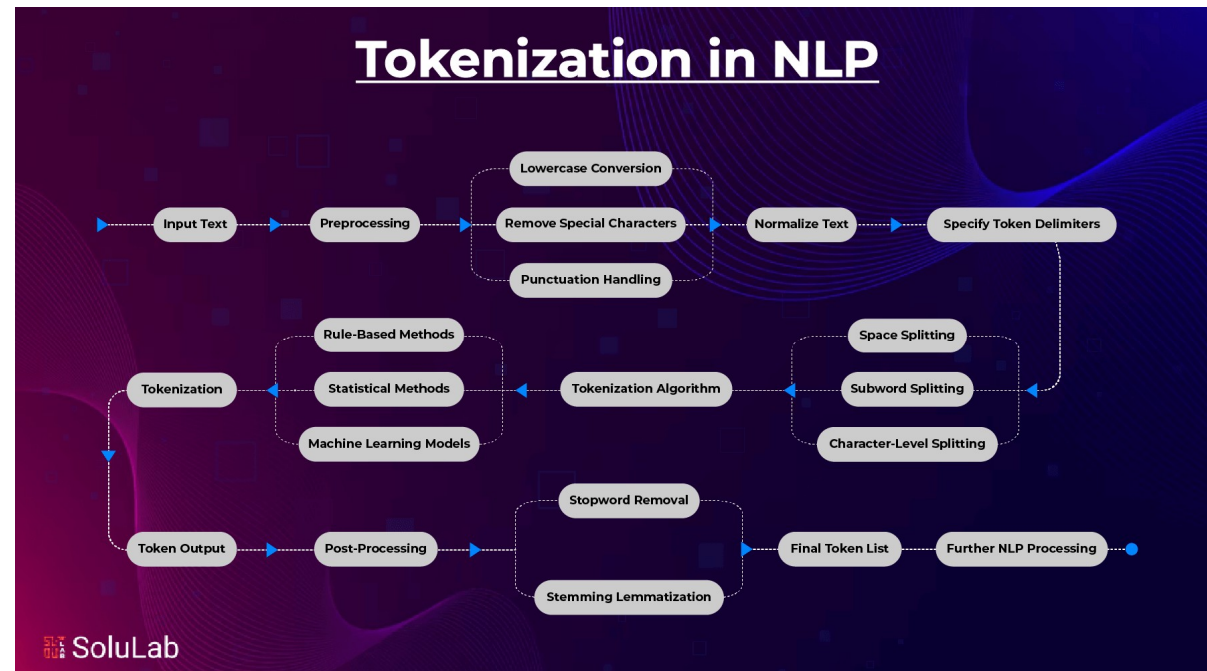
What is NLP and Why Text Is Hard

- NLP teaches computers to work with human language
- Humans understand meaning naturally
- Computers only understand numbers
- Language has ambiguity and context
- Same word can have many meanings



Raw Text → Tokens

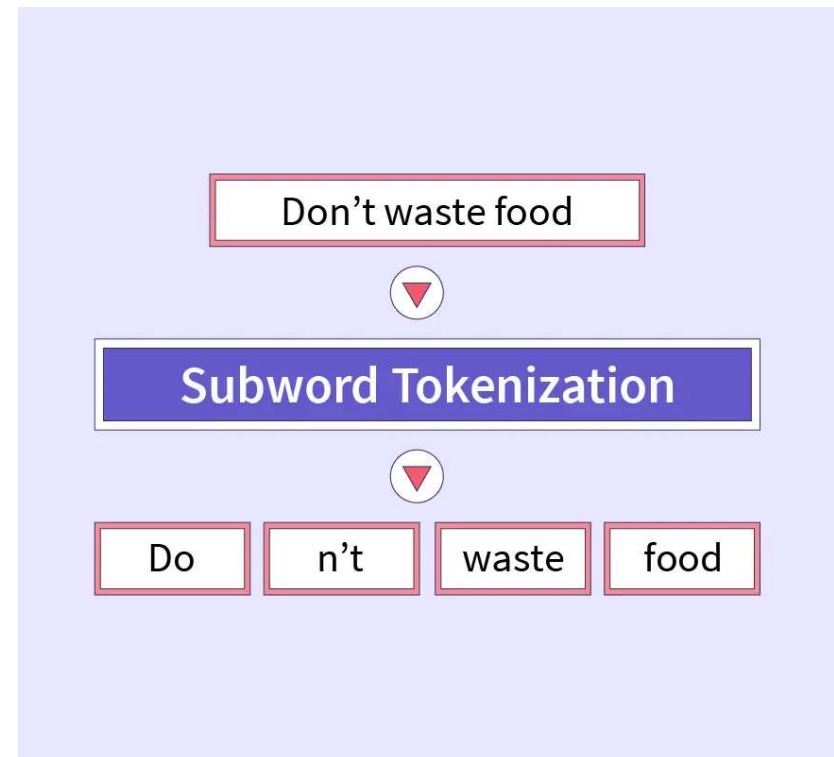
- Models cannot read raw text
- Text is split into smaller units called tokens
- Tokens are words or word pieces
- Tokenization is the first step
- Models work only with tokens



[Tokenizer Demo](#)

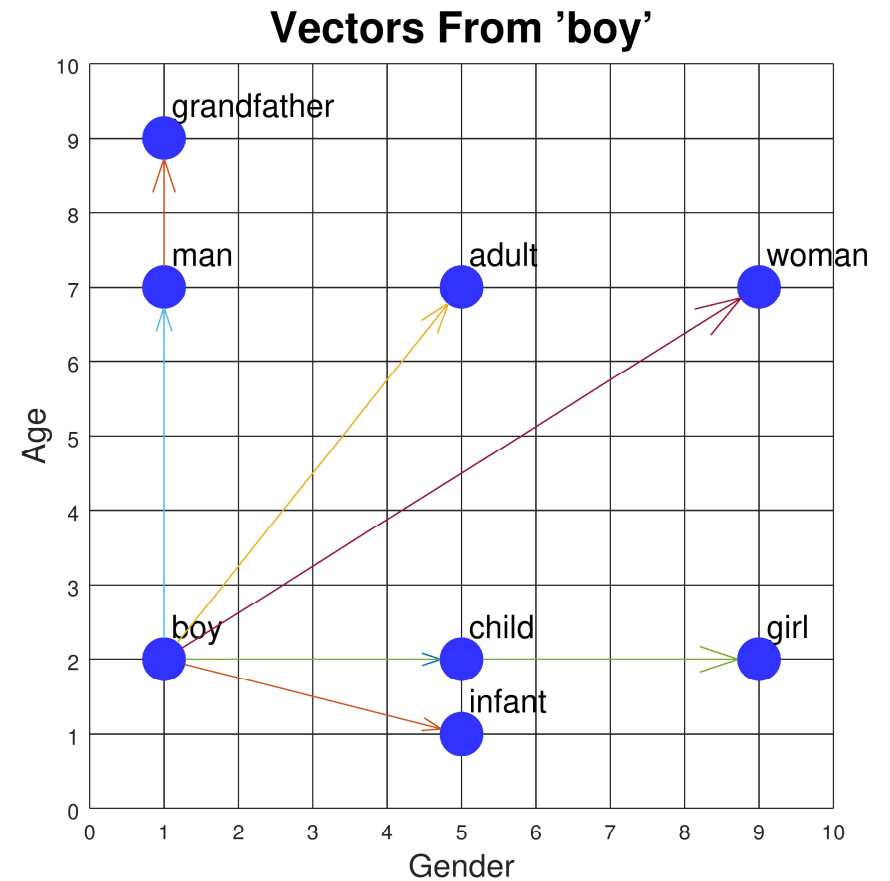
Subword Tokenization

- Words are split into smaller parts
- Common pieces are reused
- Helps handle unknown words
- Keeps vocabulary size small
- Example: unbelievable → un + believe + able



Embeddings

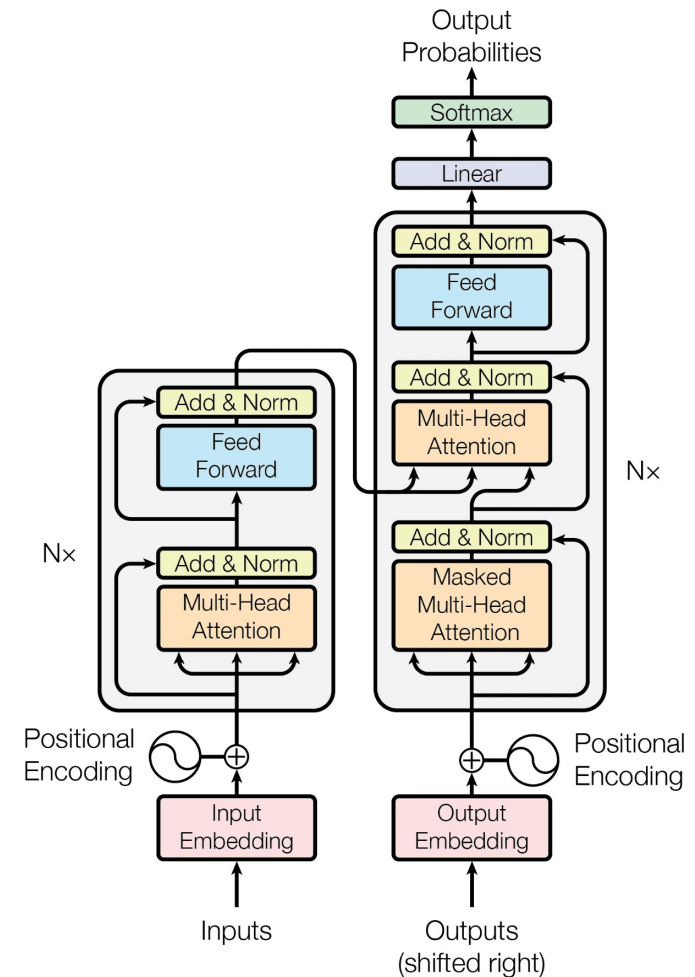
- Embeddings are numerical representations of words
- They convert text into numbers machines understand
- Each word is represented as a vector
- Similar words have similar embeddings
- Embeddings capture meaning and context



[WordEmbedding Demo](#)

What Is a Transformer

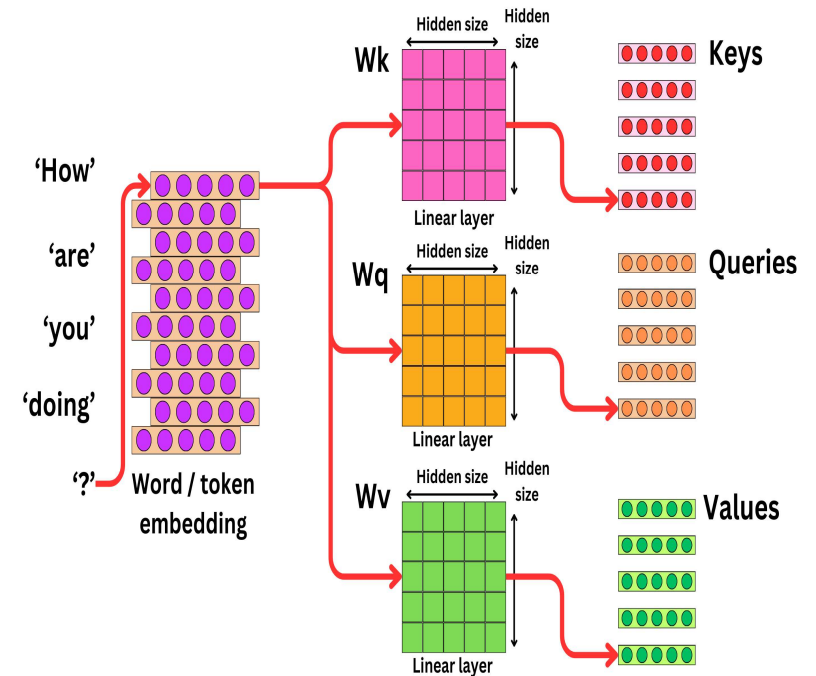
- A Transformer is a neural network architecture designed specifically to handle sequential data like text.
- Unlike older models, a Transformer processes all tokens in a sentence at the same time.
- It uses a mechanism called attention to decide which words are important for understanding context.
- This allows the model to capture long-range relationships between words.
- Transformers form the foundation of modern large language models



[transformer demo](#)

Self Attention

- Self-attention allows each token to look at every other token in the sequence at the same time.
- For each token, the model creates three vectors called Query, Key, and Value using linear transformations.
- Attention scores are computed by comparing Queries with all Keys using dot products.
- These scores are normalized to decide how much focus each token should give to others.
- A weighted sum of Values produces a context-aware representation for each token.



Key Value & Query matrix

Query (Q)

- Generated from the current word.
- Represents the information this word wants.
- Used to compare against all Keys.
- Acts like a search request.

Key (K)

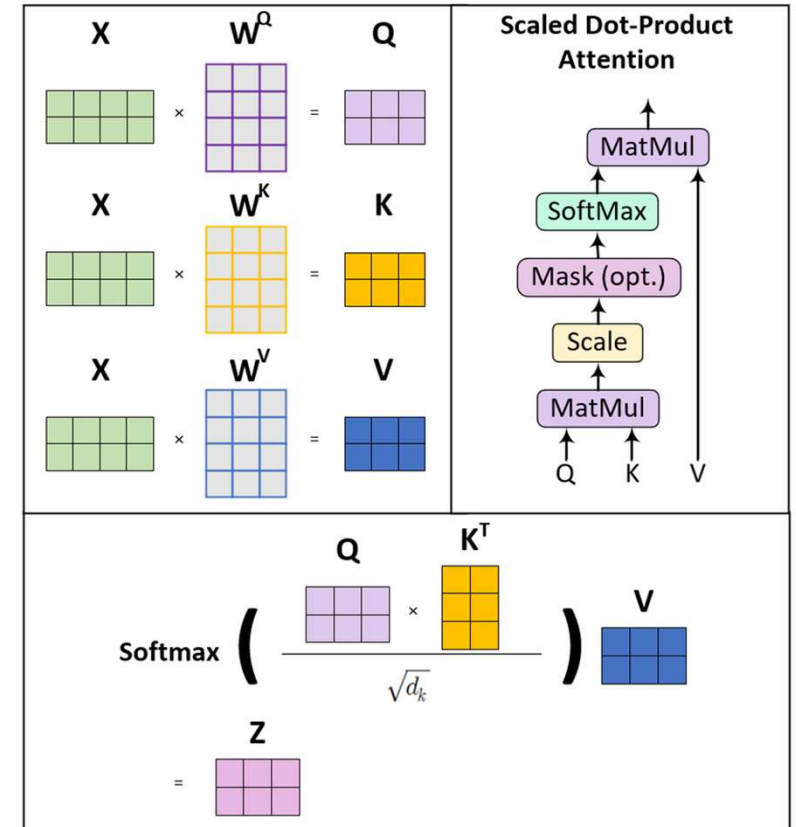
- Generated from every word in the sentence.
- Describes what information each word contains.
- Queries are matched against Keys.
- Determines relevance.

Value (V)

- Contains the actual information to be shared.
- Combined using attention weights.
- Values form the final output.
- Not used for matching, only for aggregation.

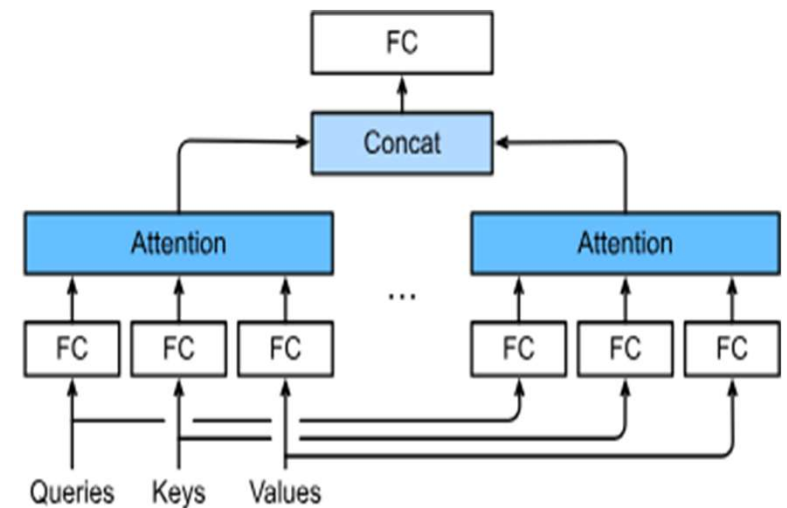
How Attention is Computed

- Query is compared with all Keys using dot product.
- Scores are normalized using softmax.
- Values are weighted and summed.
- Output is a context-aware word representation.



Multi-Head attention

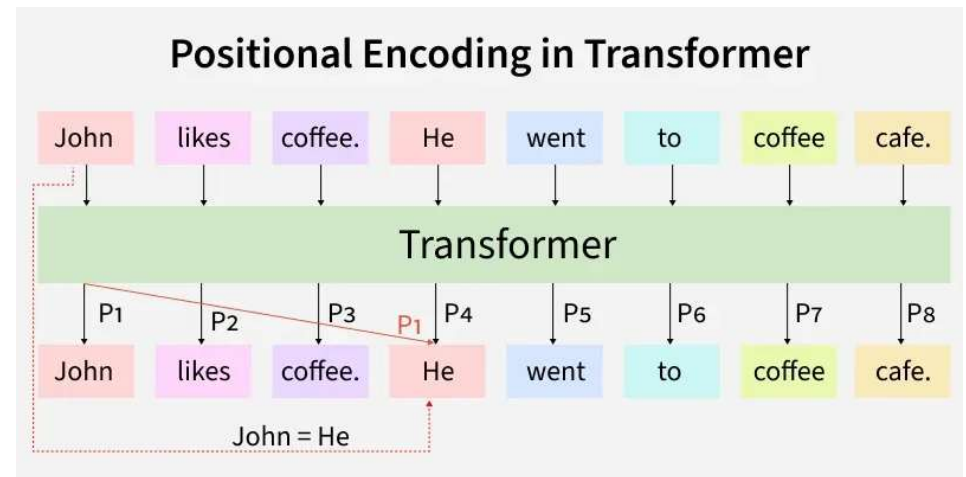
- Multi-Head Attention allows the model to focus on different parts of the sentence simultaneously.
- Instead of a single attention operation, multiple attention “heads” run in parallel.
- Each head learns a different type of relationship such as syntax, semantics, or long-range dependency.
- Outputs from all heads are concatenated and projected back to a single representation.
- This improves expressiveness without increasing sequence length



[transformer demo](#)

Positional Encoding

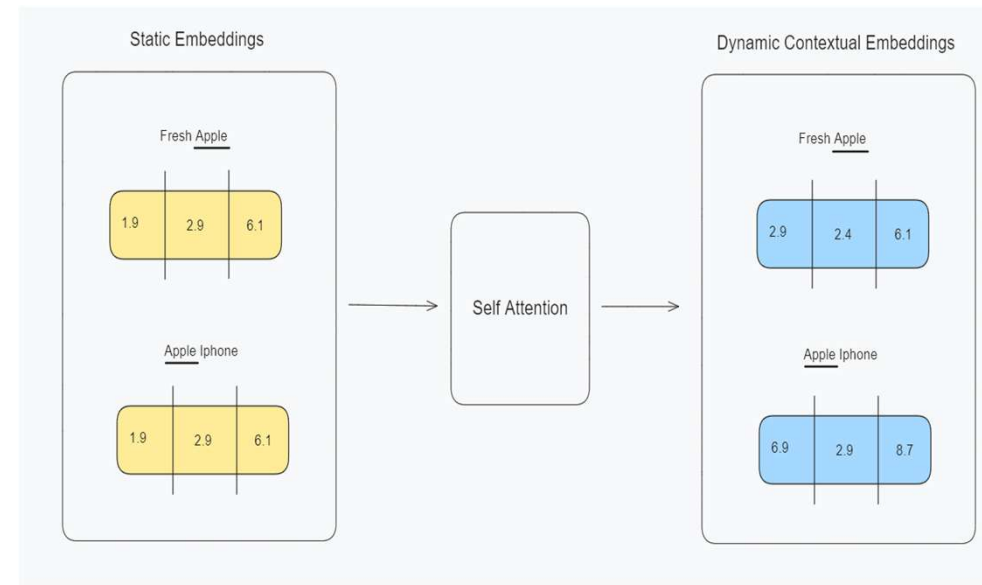
- Transformers process all tokens in parallel and do not have inherent word order.
- Positional encodings inject information about token position into embeddings.
- Typically implemented using sine and cosine functions of different frequencies.
- These encodings are added directly to word embeddings.
- This allows the model to distinguish between “dog bites man” and “man bites dog”.



[transformer demo](#)

Contextual Embeddings

- Contextual embeddings generate different vectors for the same word based on context.
- Unlike Word2Vec or GloVe, meaning is not fixed per word.
- Example: bank in “river bank” vs “bank account” gets different embeddings.
- Generated using deep models like Transformers.
- This enables true semantic understanding in modern NLP systems.



[transformer demo](#)

Encoder Architecture

- The Encoder reads the entire input sequence at once.
- It converts tokens into context-aware representations.
- Uses self-attention to relate every word with every other word.
- Positional encodings inject word order information.
- Encoder outputs are rich embeddings used for understanding tasks.

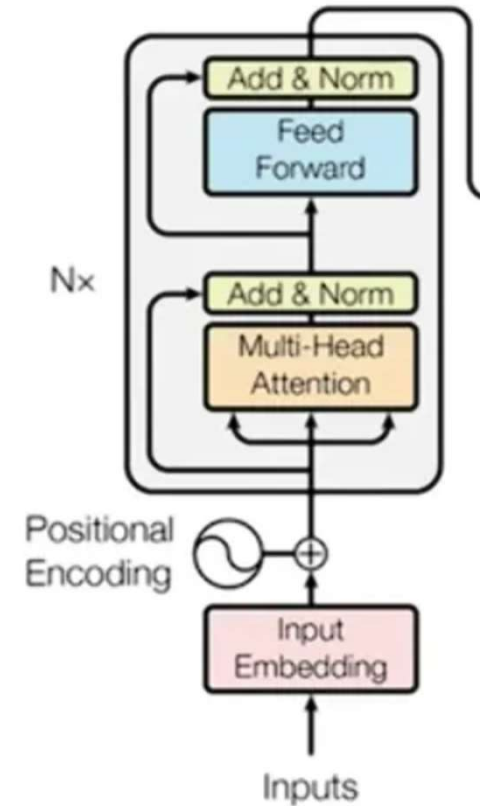
Typical use-cases

Text classification

Named Entity Recognition

Semantic search

Sentence embeddings.



[transformer demo](#)

Decoder Architecture

- The Decoder generates text token by token.
- Uses masked self-attention to prevent seeing future tokens.
- Attends to encoder outputs using cross-attention.
- Learns how to generate coherent and fluent sequences.
- Each output depends on previously generated tokens.

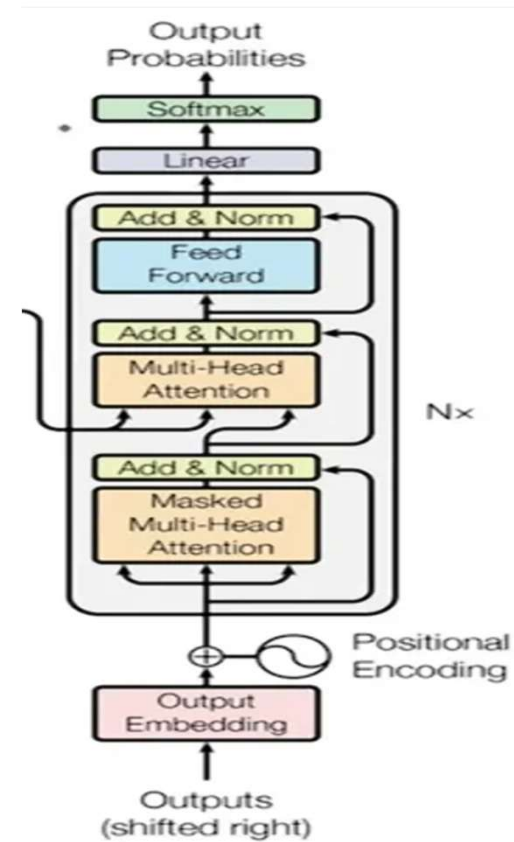
Typical use-cases

Text generation

Translation

Summarization

Chatbots



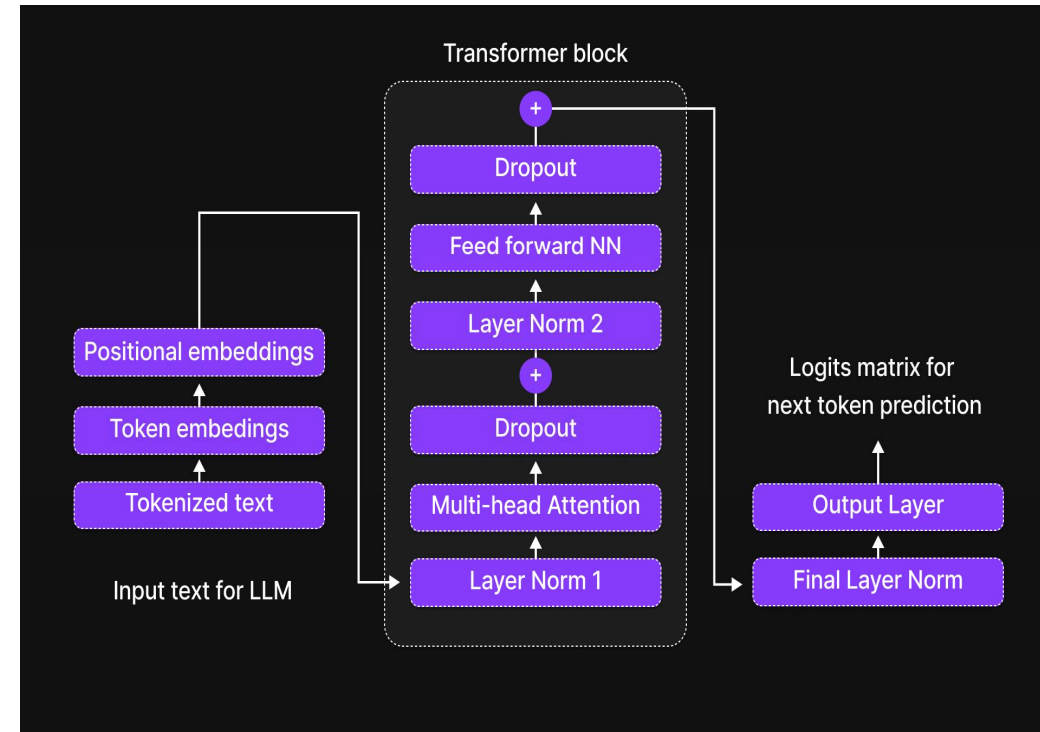
[transformer demo](#)

BERT vs GPT

Aspect	BERT	GPT
Architecture	Encoder-only	Decoder-only
Attention Type	Bidirectional	Unidirectional (causal)
Training Objective	Masked Language Modeling	Next Token Prediction
Strength	Language understanding	Language generation
Reads Text	Entire sentence at once	Left-to-right only
Typical Tasks	Classification, NER, Search	Chat, Writing, Code
Output Style	Embeddings	Generated text

Forward Pass Overview

- The forward pass is the complete journey of data from input tokens to output scores inside the Transformer.
- Tokens are first converted into embeddings so that the model can work with numbers instead of text.
- These embeddings pass through multiple layers that gradually refine the information.
- Each layer improves the model's understanding of context and relationships between words.
- The final output of the forward pass is a set of raw scores called logits.



Logits and Next Token Prediction

- Logits are numerical scores produced by the model for every possible token in the vocabulary.
- A higher logit value means the model believes that token is more likely to come next.
- Logits are not probabilities and cannot be interpreted directly by humans.
- A softmax function converts logits into probabilities that sum to one.
- The model uses these probabilities to choose the next token.

Text Generation

- The model generates text one token at a time instead of generating a full sentence at once.
- After each prediction, the new token is appended to the input sequence.
- The expanded sequence is passed through the model again.
- This process repeats until a stopping condition is met.
- Continuous repetition creates fluent and coherent text.

Pipeline API vs Low-Level API

- The pipeline API provides a simple interface that hides many implementation details.
- It is ideal for demonstrations, prototypes, and learning experiments.
- The low-level API exposes tokenization, model inputs, and outputs.
- This gives developers more control over performance and behavior.
- Production systems typically rely on low-level APIs.

Text data : Through lense of LLM

- Text is unstructured — no fixed format or schema
- LLMs don't "read" text like humans do
- Text is first tokenized into smaller units (words, subwords)
- Each token is converted into vectors (numerical form)
- These vectors capture semantic meaning and context
- Vectors flow through multiple layers in the model
- Final output is based on pattern recognition in vector space

Human : Hi ChatGPT , Myself Varun

How we think LLM sees it

'Hi ChatGPT , Myself Varun'

How LLM actually sees it

200264, 1428, 200266, 12194, 17554,
70,555, 197617, 18782, 373, 200265,
200264, 173781, 200266, 200265,
200264, 173781, 200266

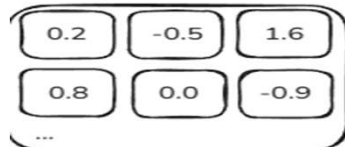
LLM Internals

neural network internals

input sequence tokens x
anywhere from 1 to e.g. 8,000 tokens



parameters (/ "weights") w
usually billions of these



giant mathematical expression

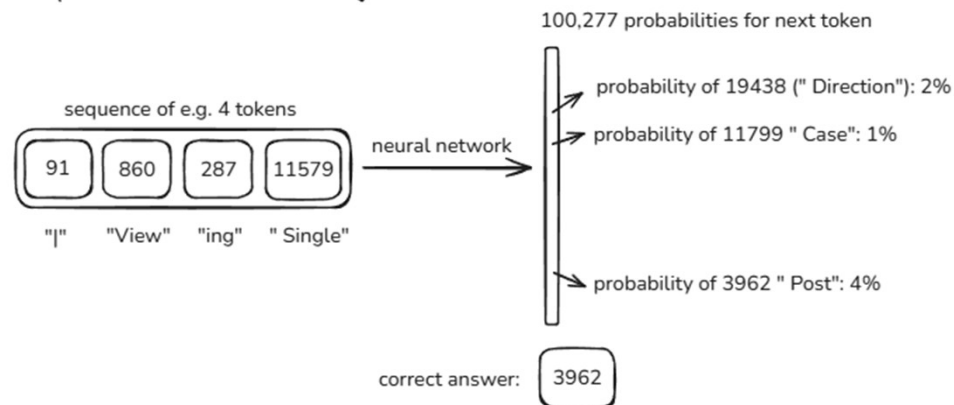
$$\frac{1}{1 + \exp(-(\mathbf{w}_0 * (1/(1 + \exp(-(\mathbf{w}_1 * x_1 + \mathbf{w}_2 * x_2 + \mathbf{w}_3)))) + \mathbf{w}_4 * (1/(1 + \exp(-(\mathbf{w}_5 * x_1 + \mathbf{w}_6 * x_2 + \mathbf{w}_7)))) + \mathbf{w}_8 * (1/(1 + \exp(-(\mathbf{w}_9 * x_1 + \mathbf{w}_{10} * x_2 + \mathbf{w}_{11})))) + \mathbf{w}_{12}))}$$

<https://bbycroft.net/llm>

100,277 numbers

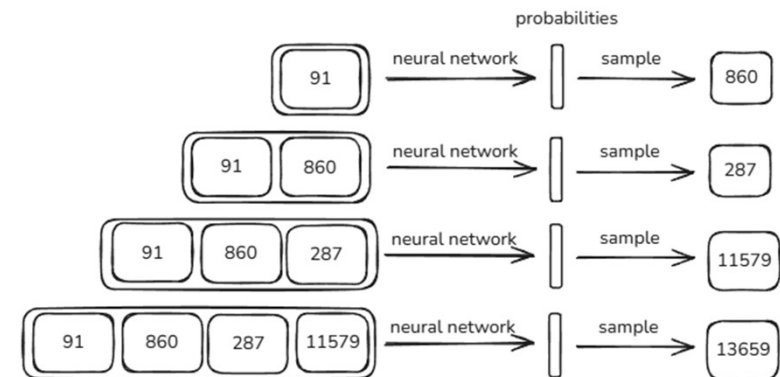


Step 3: neural network training



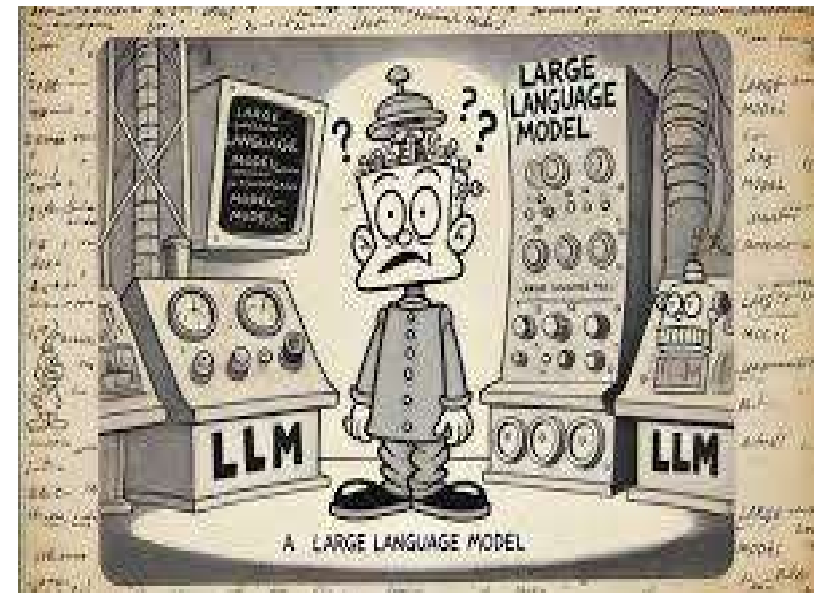
Step 4: inference

to generate data, just predict one token at a time



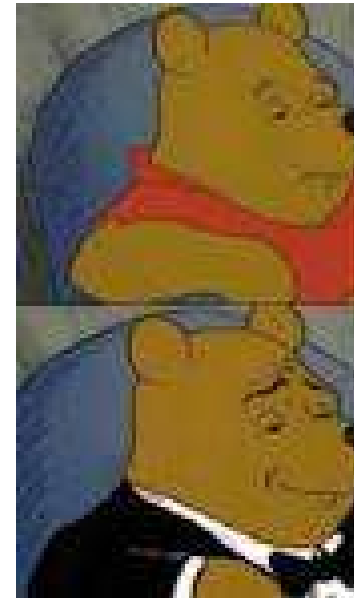
Interesting Facts about LLM

- 🧠 LLMs are not fact-checkers — they generate, not verify
- ↺ Every response is unique — no fixed output for same input
- 🎭 They can hallucinate — may create fake info with confidence
- ➕ Don't “calculate” like a calculator — rely on pattern memory
- 📄 Learn from massive data, not real-time experiences
- 📊 Use statistical correlations, not reasoning or logic
- 🔧 Not intelligent by nature — just rigorously trained models



Prompt Engineering

-  Acts as the instruction manual for LLMs
-  Helps steer output toward desired goals
-  Boosts accuracy, relevance, and reliability of responses
-  Enables complex tasks like summarization, coding, Q&A
-  Small changes in prompts can lead to big output differences
-  Bridges human intent with model interpretation
-  Foundation for building apps with LLMs (RAG, agents, etc.)

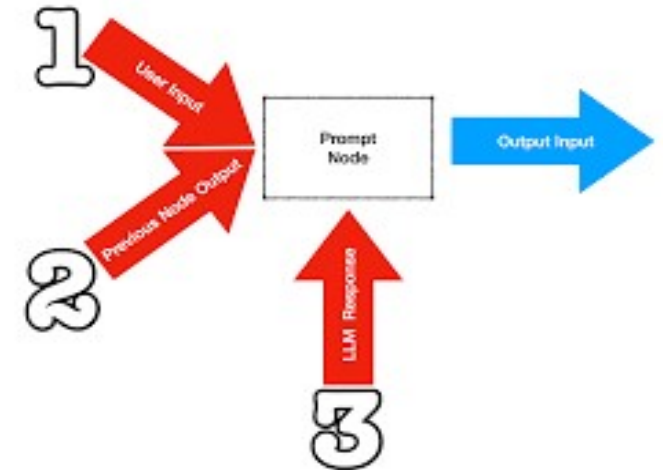


Being able to
communicate
clearly in writing

Prompt
Engineering

Prompt Drift

- 🔄 Prompt drift happens when model responses slowly go off-topic
- 🧠 LLM loses focus from the original instruction over long interactions
- 📉 Reduces output quality, accuracy, and coherence
- 📝 Caused by unclear prompts or extended dialogues
- ✅ Use few-shot examples, system prompts, or re-anchoring to fix it
- 🧩 Important in long-form chats, coding, and content generation
- 🔄 Reset or reframe the prompt periodically to regain control

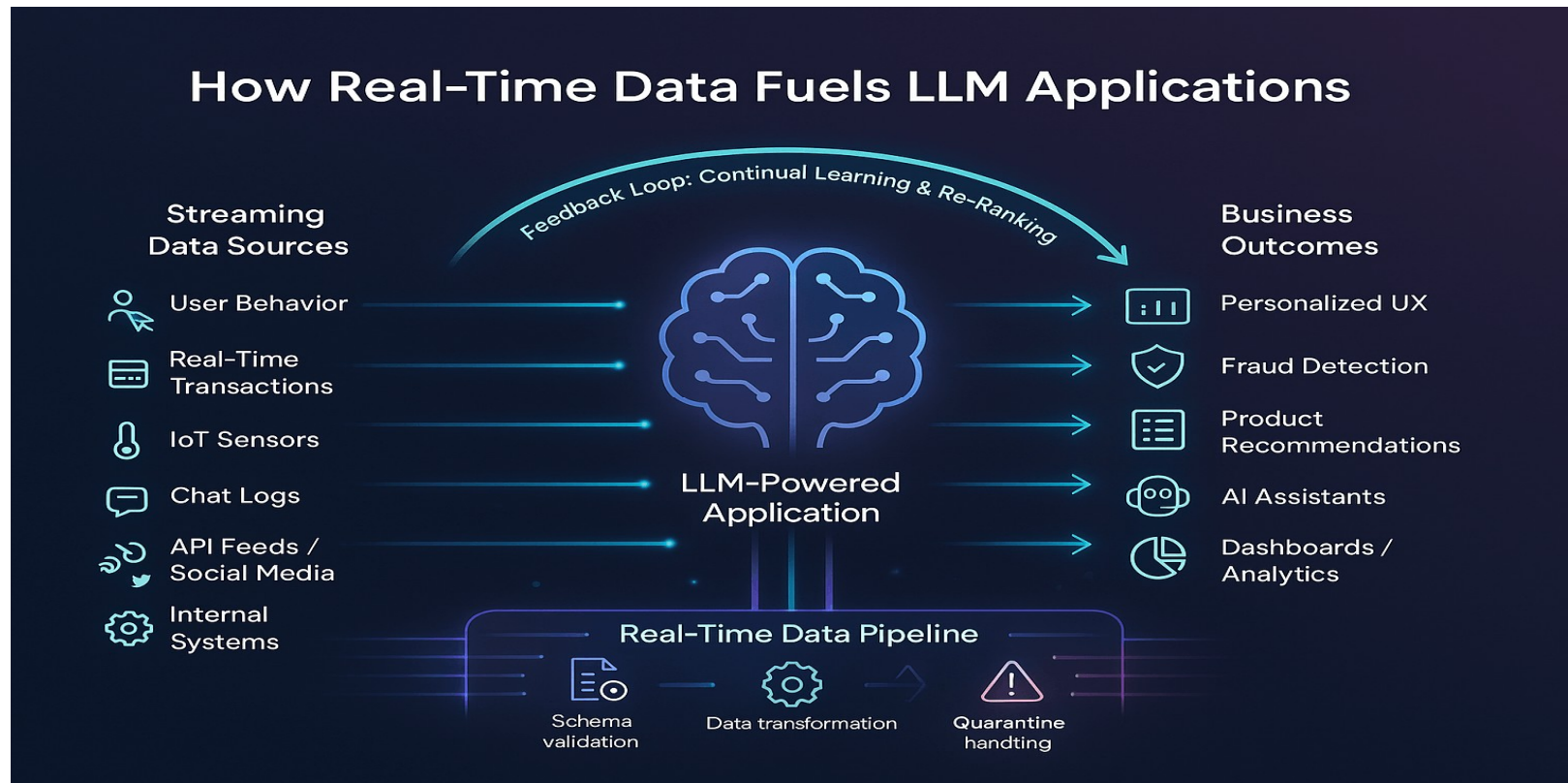


Source : <https://cobusgreyling.medium.com/prompt-drift-4873f37c43c8>

Memory-Efficient Inference

- Large language models require significant memory to run.
- GPUs offer faster inference but are costly and limited.
- CPUs are slower but more widely available.
- Quantization reduces numerical precision to save memory.
- Techniques like 8-bit loading enable efficient inference.

Real-World Use Cases



What Is Hugging Face

- **Hugging Face** is an open-source platform for working with modern AI models.
- It provides tools to use, train, and share Transformer models easily.
- Hugging Face makes complex models accessible with simple APIs.
- It is widely used in research, industry, and education.
- Many popular LLMs are available through Hugging Face.



Hugging Face Model Hub

- The Model Hub is a repository of pre-trained models.
- Models are shared for tasks like text generation and classification.
- Each model comes with documentation called a model card.
- Users can download models with just a few lines of code.
- This avoids training large models from scratch.

